SESSION 1

Greenfield with AI

From zero to working app with specs, TDD, and AI as a real collaborator

Pasi Vuorio | Founder & AI Architect, ModernPath

Today's Agenda

01 **AI Coding Tools & Project Rules** — 5 min

Cursor, Copilot, Claude Code, Codex -- choose the right tool, then configure it

02 **Skills -- Making Capabilities Discoverable** — 10 min

Document what the AI can do so it actually does it

03 **Effective Prompting & Model Selection** — 10 min

Context sandwich, @ injection, match brain to task

04 **Spec-Driven Development & Acceptance Criteria** — 10 min

Specs before code -- user acceptance criteria that AI can test

05 **TDD with AI + Two End-to-End Builds** — 35 min

Red -> Green -> Refactor on your own project

AI Coding Tools Landscape

Each tool has a sweet spot -- choose based on the task at hand

Cursor

Multi-model quick iterations, tasks and debugging. Composer 2 is the fastest coding model available.

Best for: Fast iteration, multi-file edits

Claude Code

Best agentic coding tool. Planning and implementing long-running coding tasks with full reasoning.

Best for: Architecture, complex implementation

GitHub Copilot

Enterprise compatible tool. Tight VS Code and GitHub integration. Lacks some capabilities vs competitors.

Best for: Enterprise environments

Codex CLI

Tool for hard programming tasks and solving problems. Terminal-native, GPT-5.4 powered.

Best for: Hard problems, automation

Rules & Configuration

Your codebase constitution -- processes, links, and guardrails

Project Structure

```
project-root/
  .cursor/
    rules/            # Cursor rules
      main.mdc        # Always on
      testing.mdc     # For tests
  CLAUDE.md           # Claude Code
  AGENTS.md           # Cross-tool
```

What Rules Should Define

- Processes (spec-driven, TDD, review)
- Index to tools, docs, and skills
- Basic coding styles and conventions
- Testing instructions and frameworks
- Guardrails -- what NOT to do

Rehearsal: Clone & Start

Clone the training repo, then prompt a quick intent spec using ready examples

```
git clone https://github.com/pasilastbot/ai_training
```

Steps

1. Clone the repo and explore the structure
2. Prompt a quick intent spec using examples
3. Build the spec before implementing

What's in the Repo

CLAUDE.md + AGENTS.md templates

prompts/01-scaffold.md .. 04-feature2.md

Example specs and acceptance criteria

10 min

Discoverable Skills

If it's not documented, the AI doesn't know it exists

What is a Skill?

A documented capability the AI can discover and invoke.

Without documentation, your scripts, APIs, and tools are invisible to the assistant. A skill block in AGENTS.md is the handoff.

AGENTS.md Skill Block

```
## skill: run-tests
description: Run the test suite
trigger: "test", "verify", "check"
command: npm test
output: test results + coverage

## skill: lint
description: Check code style
command: npx eslint src/
```

Skills = What the AI can do | Sub-Agents = Who does it | Hooks = When to verify (Sessions 2-3)

Effective Prompting: Context Sandwich

Evidence -> Request -> Constraints

TOP BREAD -- Context

```
@src/auth/login.ts
@docs/architecture.md
```

FILLING -- Request

```
"Refactor the login function to
use the new OAuth2 flow
documented in architecture.md"
```

BOTTOM BREAD -- Constraints

```
"Keep existing error handling.
Add unit tests for the new flow."
```

@ Context Injection

`@file`	Include specific file
`@folder`	Include folder contents
`@codebase`	Semantic search across project
`@docs`	Include documentation
`@web`	Search the internet
`@git`	Include git context

Model Selection

Match the brain to the task -- different strengths for different jobs

Model	Reasoning	Speed	Context	Best For
Claude Opus 4.6	Best	Slow	1M	Architecture, complex reasoning, planning
Claude Sonnet 4.6	Great	Fast	1M	Daily coding, balanced cost + quality
GPT-5.4	Best	Medium	1M+	Hard tasks, computer use, reasoning
Gemini 3.1 Pro	Best	Medium	1M+	Massive codebases, long documents, visual reasoning
Composer 2	Great	Fastest	200K	Fastest coding model, multi-file edits, quick fixes, documentation

Sonnet for daily work -> Opus/GPT-5.4 for deep reasoning -> Composer 2 for fast iteration

Build: Scaffold Your Project

Write your first Context Sandwich prompt using prompts/01-scaffold.md

1. **Provide context**
   Your CLAUDE.md, package.json, project conventions

2. **Request**
   Scaffold the project -- folder structure, types, framework

3. **Constraints**
   Follow the project rules you defined in CLAUDE.md

AI generates: folder structure, TypeScript types, CLI/API framework

10 min

Spec-Driven Development

The spec IS the prompt -- write the contract before the code

1 **Define Spec** → 2 **Write Tests** → 3 **Implement Code** → 4 **Validate & Ship**

Vibe coding = "make it work, hope for the best" -- dangerous in production

Spec-driven = "define what right looks like, then build to that standard"

A good spec = a good AI output. Without it, you're hoping the AI guesses correctly.

User Acceptance Criteria

Define what 'done' means so AI can test it

Feature: Add Todo Item

AC1: Adding a valid item

- User submits "Buy groceries"
- List contains 1 item, status "pending"
- Success message displayed

AC2: Empty input rejected

- User submits empty string
- Error "Title required" returned
- List remains unchanged

AC3: Duplicate detection

- Submitting same title twice is rejected
- Original item remains unchanged

"Can AI Test This?" Check

- Is every AC measurable? (numbers, states)
- Are edge cases explicit? (empty, null, errors)
- Are side effects listed? (logs, DB writes)
- Is the happy path AND error path defined?
- Can each AC become a test case directly?

Build: Write Your Feature Spec

Use prompts/02-spec.md -- the spec becomes your AI prompt for implementation

1. Write acceptance criteria for Feature 1 of your project
2. Include happy path, error cases, and edge cases
3. Run the "Can AI Test This?" check on each AC

10 min

TDD with AI

Red -> Green -> Refactor -- the test proves AI wrote what you asked for

RED

Write a failing test from your spec's AC. YOU define the contract.

→

GREEN

Ask AI to write the minimum code to make the test pass.

→

REFACTOR

Ask AI to improve code quality. Keep all tests passing.

TDD + AI = confidence. The test proves the AI wrote what you asked for, not what it hallucinated.

Build: Implement with TDD

Feature 1 with TDD, then spec + build Feature 2

Feature 1

Using prompts/03-tdd.md:

1. Write tests first from your ACs
2. AI generates implementation
3. Run tests -> all green
4. Refactor together

Feature 2

Using prompts/04-feature2.md:

1. Write spec independently
2. Drive the full TDD cycle
3. Review output against ACs
4. Ship with confidence

15 min

Key Takeaways

- Configure your AI with CLAUDE.md / AGENTS.md -- processes, links, and guardrails
- Document skills -- if the AI can't find it, it can't use it
- Context Sandwich -- always ground prompts with evidence + request + constraints
- Match model to task -- Opus for thinking, Sonnet for coding, Composer 2 for speed
- Spec first -- acceptance criteria that AI can turn directly into test cases
- TDD with AI -- Red -> Green -> Refactor gives you confidence, not hope

Your workflow: Rules -> Skills -> Scaffold -> Spec -> TDD -> Ship